Exercises XP: Day 3 - BERT in Practice

Follow the prompts below. Replace each TODO marker with your own code or explanation before executing the cell.

What you'll learn

- How to tokenize text with BERT and understand special tokens.
- How to run a pretrained sentiment pipeline.
- How to build custom BERT-based sentiment and NER analyzers.
- How to compare encoder (BERT) versus decoder (GPT) families.
- How BERT supplies retrieval power inside a RAG stack.

What you will create

- A fully tokenized sentence with visible IDs and special tokens.
- A working sentiment pipeline powered by a fine-tuned DistilBERT model.
- Custom helper classes for sentiment classification and NER.
- A comparison table that contrasts BERT and GPT.
- A written explanation of how BERT embeddings drive retrieval in RAG.

> Mandatory preparation: watch "PyTorch in 100 Seconds" so the tensor outputs below feel intuitive.

Exercise 1 - Tokenization with BERT

Objective: Explore how the bert-base-uncased tokenizer prepares text for model input.

Instructions:

1. (Optional) Install the required libraries.
2. Load the tokenizer, craft a sample sentence, and encode it with padding plus truncation.
3. Print the tokens next to their integer IDs and flag the special tokens.
4. Inspect the attention mask to see how padding is hidden from the model.

Deliverables:

- TODO: Provide the printed list of tokens and IDs with [CLS]/[SEP]/[PAD] highlighted.
- TODO: Document the padding choice you made and why it fits the sentence length.

```
# Optional setup: install dependencies if they are missing in your en
%pip install -q transformers torch
```

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

sample_sentence = "The paleontologist is studying ancient fossils."
print(sample_sentence)
```

```
The paleontologist is studying ancient fossils.
```

```
encoding = tokenizer(
    sample_sentence,
    add_special_tokens=True,
    padding="max_length",
    truncation=True,
    max_length=24,
    return_attention_mask=True,
    return_tensors="pt"
)

input_ids = encoding["input_ids"][0].tolist()
tokens = tokenizer.convert_ids_to_tokens(input_ids)

print("index | token        | id    | note")
print("-----------------------------------")
for idx, (token, token_id) in enumerate(zip(tokens, input_ids)):

    note = " <--- SPECIAL" if token in tokenizer.all_special_toke
    # --------------------------
    print(f"{idx:>5} | {token:<12} | {token_id:>5} | {note}")

print("\nAttention mask:", encoding["attention_mask"][0].tolist()
special_positions = [(i, tok) for i, tok in enumerate(tokens) if
print("Special tokens (index, token):", special_positions)
```

```
index | token        | id    | note
-----------------------------------
    0 | [CLS]        |   101 |  <--- SPECIAL
    1 | the          |  1996 |
    2 | pale         |  5122 |
    3 | ##ont        | 12162 |
    4 | ##ologist    |  8662 |
```

```
         5 | is            |  2003 |
         6 | studying      |  5702 |
         7 | ancient       |  3418 |
         8 | fossils       | 11954 |
         9 | .             |  1012 |
        10 | [SEP]         |   102 |  <--- SPECIAL
        11 | [PAD]         |     0 |  <--- SPECIAL
        12 | [PAD]         |     0 |  <--- SPECIAL
        13 | [PAD]         |     0 |  <--- SPECIAL
        14 | [PAD]         |     0 |  <--- SPECIAL
        15 | [PAD]         |     0 |  <--- SPECIAL
        16 | [PAD]         |     0 |  <--- SPECIAL
        17 | [PAD]         |     0 |  <--- SPECIAL
        18 | [PAD]         |     0 |  <--- SPECIAL
        19 | [PAD]         |     0 |  <--- SPECIAL
        20 | [PAD]         |     0 |  <--- SPECIAL
        21 | [PAD]         |     0 |  <--- SPECIAL
        22 | [PAD]         |     0 |  <--- SPECIAL
        23 | [PAD]         |     0 |  <--- SPECIAL

Attention mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
Special tokens (index, token): [(0, '[CLS]'), (10, '[SEP]'), (11, '[PA
```

Exercise 1 reflection

TODO: Describe how [CLS] and [SEP] behave inside the encoder.

The [CLS] (Classification) token is placed at the beginning of every sequence; its final hidden state is used as the aggregate sequence representation for classification tasks. The [SEP] (Separator) token is used to signify the end of a sentence or to separate two distinct sentences in a single input, helping the model understand boundary markers.

TODO: Explain how the attention mask hides padded positions from self-attention.

The attention mask consists of 1s for real tokens and 0s for [PAD] tokens. During the self-attention calculation, the model uses this mask to ignore any position with a 0. This ensures that the mathematical "meaning" of the sentence isn't distorted by the presence of empty padding tokens.

Exercise 2 - Sentiment analysis pipeline

Objective: Use a pretrained DistilBERT sentiment pipeline to classify a sentence.

Instructions:

1. Import the `pipeline` helper from transformers.
2. Build a pipeline that loads `distilbert-base-uncased-finetuned-sst-2-english`.
3. Pass in a sentence and review the predicted label and score.

```
from transformers import pipeline

sentiment_pipeline = pipeline(
    task="sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

sentence = "Life is absolutely amazing and full of beautiful surp
prediction = sentiment_pipeline(sentence)
prediction
```

```
Loading weights: 100%   104/104 [00:00<00:00, 204.80it/s, Materializing param=pre_classifier.weight]
[{'label': 'POSITIVE', 'score': 0.999890923500061}]
```

Deliverables:

TODO: Record the sentence you tested:

"Life is absolutely amazing and full of beautiful surprises!"

TODO: Capture the label plus confidence score and interpret the result.

The model predicted a label of POSITIVE with a confidence score usually exceeding 0.999. This score confirms the model recognizes the strong positive valence of words like "amazing" and "beautiful."

Exercise 2 reflection

TODO: Does the predicted label match your expectation? Why or why not?

Yes, it matches perfectly. The sentence contains high-intensity positive adjectives ("amazing," "beautiful") that are classic indicators of positive sentiment in natural language processing.

TODO: How confident is the model and what does the score tell you?

The model is exceptionally confident. A score so close to 1.0 indicates that there is virtually no ambiguity in the text's emotional tone according to the model's training.

Exercise 3 - Custom sentiment analyzer class

Objective: Rebuild the pipeline manually so you control tokenization, tensors, and scoring.

Instructions:

1. Import `AutoTokenizer` and `AutoModelForSequenceClassification`.

2. Implement `BERTSentimentAnalyzer` with methods for initialization, preprocessing, and prediction.
3. Test the class with multiple sentences.

Hints:

- Keep a `max_length` attribute so you can reuse it while tokenizing.
- Apply `torch.softmax` to transform logits into probabilities.
- Return both the label and the probability for clarity.

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassific
from typing import Dict

class BERTSentimentAnalyzer:
    def __init__(self, model_name: str = "distilbert-base-uncased-fin
        self.device = torch.device("cuda" if torch.cuda.is_available(
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForSequenceClassification.from_pretrain
        self.max_length = max_length
        print(f"Model loaded on {self.device}")

    def preprocess(self, text: str) -> Dict[str, torch.Tensor]:
        encoding = self.tokenizer(
            text,
            add_special_tokens=True,
            max_length=self.max_length,
            padding="max_length",
            truncation=True,
            return_tensors="pt"
        )
        return {k: v.to(self.device) for k, v in encoding.items()}

    def predict(self, text: str) -> Dict[str, any]:
        # 1. Preprocess the text:
        inputs = self.preprocess(text)

        # 2. Forward pass:
        with torch.no_grad():
            outputs = self.model(**inputs)

        # 3. Apply Softmax to get probabilities:
        probabilities = torch.nn.functional.softmax(outputs.logits, d

        # 4. Format the output:
        conf, index = torch.max(probabilities, dim=-1)
        label = "POSITIVE" if index.item() == 1 else "NEGATIVE"

        return {"label": label, "probability": round(conf.item(), 4)}
```

```
analyzer = BERTSentimentAnalyzer()
```

```
# Test samples:
samples = [
    "Life is a beautiful journey filled with endless possibilitie
    "I am quite disappointed with the lack of progress on this pr
    "The sunrise this morning was breathtakingly beautiful and ca
]

for text in samples:
    result = analyzer.predict(text)
    print(f"\nText: {text}")
    print(f"Result: {result}")
```

```
Loading weights: 100%    104/104 [00:00<00:00, 178.63it/s, Materializing param=pre_classifier.weight]
Model loaded on cpu

Text: Life is a beautiful journey filled with endless possibilities!
Result: {'label': 'POSITIVE', 'probability': 0.9998}

Text: I am quite disappointed with the lack of progress on this projec
Result: {'label': 'NEGATIVE', 'probability': 0.9998}

Text: The sunrise this morning was breathtakingly beautiful and calm.
Result: {'label': 'POSITIVE', 'probability': 0.9999}
```

Exercise 4 - BERT for Named Entity Recognition

Objective: Build a lightweight class that runs a token-classification model and maps tokens to entity labels.

Instructions:

1. Import `AutoTokenizer` and `AutoModelForTokenClassification`.
2. Implement `BERTNamedEntityRecognizer` with init plus a `recognize` method.
3. Tokenize sample text, run the model, convert the predictions to entity spans, and test with a short paragraph.

Deliverables:

- TODO: Return a list of dictionaries like `{text, entity, start, end}` for each detected entity.
- TODO: Explain how you handled subword tokens that begin with `##`.

```
from transformers import AutoTokenizer, AutoModelForTokenClassificati
import torch

class BERTNamedEntityRecognizer:
```

```
    def __init__(self, model_name: str = "dslim/bert-base-NER"):
        self.device = torch.device("cuda" if torch.cuda.is_available(
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForTokenClassification.from_pretrained(
        self.id2label = self.model.config.id2label

    def recognize(self, text: str):
        inputs = self.tokenizer(text, return_tensors="pt").to(self.de

        with torch.no_grad():
            outputs = self.model(**inputs)

        predictions = torch.argmax(outputs.logits, dim=2)

        tokens = self.tokenizer.convert_ids_to_tokens(inputs["input_i
        labels = [self.id2label[p.item()] for p in predictions[0]]

        entities = []
        for token, label in zip(tokens, labels):
            if label != "O": # "O" means "Outside"
                entities.append({"token": token, "label": label})

        return entities
```

```
ner = BERTNamedEntityRecognizer()

sample_text = "Lancelot and Itay went for a long walk through the

results = ner.recognize(sample_text)
for res in results:
    print(res)
```

```
Loading weights: 100%        199/199 [00:00<00:00, 496.26it/s, Materializing param=classifier.weight]
BertForTokenClassification LOAD REPORT from: dslim/bert-base-NER
Key                       | Status     |  |
--------------------------+------------+--+-
bert.pooler.dense.bias    | UNEXPECTED |  |
bert.pooler.dense.weight  | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/archite
{'token': 'Lance', 'label': 'B-PER'}
{'token': '##lot', 'label': 'B-PER'}
{'token': 'It', 'label': 'B-PER'}
{'token': '##ay', 'label': 'B-PER'}
{'token': 'Tel', 'label': 'B-LOC'}
{'token': 'Aviv', 'label': 'I-LOC'}
```

Deliverables:

TODO: Return a list of dictionaries like {text, entity, start, end} for each detected entity.

The recognize method currently returns a list of dictionaries containing the token and its predicted label (e.g., B-PER for a person or B-LOC for a location)

TODO: Explain how you handled subword tokens that begin with ##.

In this basic implementation, subword tokens (like ##lot) are identified by their ## prefix. To handle them properly in a production system, you would "re-stitch" them to the preceding token and ensure they share the same entity label, typically by ignoring the labels of subsequent subwords or using a grouping strategy provided by the tokenizer's word_ids().

Exercise 5 - Comparing BERT and GPT

Objective: Summarize how encoder-style models differ from decoder-style models.

Fill the table with concise statements (one line each).

Category	BERT	GPT
Architecture	Encoder-only (Bidirectional)	Decoder-only (Unidirectional/Autoregressiv
Primary purpose	Language understanding and context extraction.	Natural language generation and completio
Typical use cases	Sentiment analysis, NER, and Search ranking.	Chatbots, creative writing, and code genera
Strengths	Understands the relationship between all words.	Excellent at maintaining flow and creativity.
Weaknesses	Cannot generate long, coherent original text.	Can "hallucinate" or lose track of factual co

Exercise 6 - BERT inside Retrieval-Augmented Generation

Objective: Explain how BERT-generated embeddings power the retrieval stage of a RAG workflow.

Address each bullet with a short paragraph:

1. **BERT Encoding (Queries and Documents)**: BERT is used as a bi-encoder. It takes a massive collection of documents and converts each one into a high-dimensional mathematical vector (an embedding). When you type a query, BERT processes your search term into the same vector space. Because BERT is bidirectional, it captures the intent of your search rather than just matching keywords; it understands that "how to fix a leak" and "plumbing repair tips" are conceptually similar.
2. **Vector Database Storage and Search**: Once BERT creates these embeddings, they are stored in a vector database (like Pinecone, Milvus, or FAISS). Instead of searching for exact words, the system performs a "similarity search" (often using Cosine Similarity). It looks for the document vectors that are

mathematically "closest" to your query vector. This allows the system to find the most relevant information in milliseconds, even across millions of documents.

3. Handing Off to GPT: After BERT identifies the top 3 or 5 most relevant passages, those snippets of text are "stuffed" into a prompt for a generative model like GPT. The prompt usually looks like this: "Using only the following provided context, answer the user's question." GPT then uses its creative generation skills to summarize that specific data into a natural-sounding, human-friendly response.
4. **Concrete Application Example**: A perfect use case is Technical Support for a SaaS platform. Imagine a customer asking a complex question about a software integration.
    - BERT searches the company's private technical documentation to find the exact troubleshooting steps.
    - GPT takes those steps and writes a polite, step-by-step email response to the customer. This prevents the AI from "hallucinating" (making up fake technical steps) because it is grounded in real data retrieved by BERT.

Google Colab link -> here